

OOPJ Day 4

Thu, 10 Sept 26

Debugging in Eclipse: Breakpoints and Function Keys

- Four essential function keys to memorize:
 - F11: start the debugger (launches debug mode)
 - F5: step into a function (enters the body of a user-defined function line by line)
 - F6: step over (executes the current line entirely and moves to the next)
 - F8: resume execution and jump to the next breakpoint
 - Ctrl+F2 or the Terminate button: stop the debug session
- Setting breakpoints:
 - Double-click on the line number in the left margin to toggle a breakpoint
 - Alternatively: Run menu > Toggle Breakpoint
 - Set a breakpoint at the line where you suspect the problem begins, not before
 - Example: if lines 1–7 work correctly but line 8 may have an issue, set the breakpoint at line 8
- Stepping into functions:
 - Use F5 only when you want to enter a user-defined function and trace it line by line
 - For predefined/library functions, use F6 to execute them without entering their internals
 - Example: stepping into `getIn()` with F5 will enter the function body; F6 will execute it and return
- Debug perspective:
 - On first debug launch, Eclipse prompts to switch to the Debug perspective; click Switch
 - The green highlighted line is the line about to be executed (not yet run)
 - Variables window (right side) shows all live variables and their current values
 - Expand object variables to inspect individual data member values
 - To return to normal coding view: click the Java perspective icon (top-right) or Window > Open Perspective > Java
- Multiple breakpoints:
 - Set a second breakpoint at a later line (e.g., line 11) to skip over already-verified code
 - Press F8 to jump directly from the first breakpoint to the next one
 - Use this to avoid stepping through lines you already know are correct

Java Data Types: Primitive vs. Object

- Two primary categories of data types in Java:

- Primitive types
- Object types (reference types)
- Primitive data types:
 - Predefined, inbuilt language data types
 - Memory is automatically and directly allocated for them
 - In most cases, allocated on the stack (specifically when declared as local variables)
 - Full list: byte, short, int, long, float, double, boolean, char
 - Example: `int num1` declared inside `main()` gets 4 bytes on the stack of `main` at, say, address 200
 - The compiler uses a symbol table to map variable names to memory addresses
 - Symbol table entries: variable name + memory address
 - When `num1` is used in code, the JVM looks up the symbol table and substitutes the address
- Object (reference) types:
 - User-defined data types; an object is an instance of a class
 - Example: `Calculator` is a class; `Calculator obj = new Calculator()` creates an instance
 - Objects are always allocated on the heap in Java, never on the stack
 - This is a fundamental, non-negotiable rule in Java
- Why primitives inside objects go to the heap:
 - If a class `Calculator` has `int x`, `int y`, `int z`, `int` memory as data members
 - When `new Calculator()` is called, the entire object (including its primitive fields) is placed on the heap
 - So primitive types are on the stack only when declared as standalone local variables
 - If they are data members of a class, they follow the object to the heap
- Stack vs. heap clarification:
 - Stack and heap are not physically separate memory regions; they are the same physical RAM
 - The distinction is purely for classification: life, scope, and management differ
 - Stack variables: limited life and scope (destroyed when the function returns)
 - Heap variables: persist until explicitly deleted (C++) or garbage collected (Java)
 - Java's garbage collector (a JVM module) automatically reclaims heap memory no longer in use

Memory Model: References vs. Pointers

- C++ pointer recap:
 - `int num1` at address 200, value 5
 - `int* ptr = &num1` creates a 4-byte pointer at address 300, holding value 200
 - `*ptr` dereferences to get 5
 - `ptr++` performs pointer arithmetic: `ptr` now holds 204 (moves by `sizeof(int) = 4` bytes)

- Accessing address 204 if it doesn't belong to the program causes a crash or OS-level violation
- Java reference vs. C++ pointer:
 - Java reference (e.g., Calculator obj) is conceptually identical to a C++ pointer
 - Both hold the memory address of an object
 - Key difference: Java references cannot be arithmetically manipulated
 - obj++ is a compile-time error in Java
 - This is intentional: Java prevents memory manipulation to ensure security
 - The term "reference" was chosen deliberately to avoid confusion with C++ pointers that support arithmetic
- C++ two-style object creation (recap for contrast):
 - Calculator obj; → object created on the stack of the function; destroyed when function ends
 - Calculator* ptr = new Calculator(); → pointer on stack, object on heap; persists until delete ptr
- Java object creation:
 - Calculator obj; → this does NOT create an object; it creates a 4-byte reference on the stack, equivalent to Calculator* ptr in C++
 - Calculator obj = new Calculator(); → reference on stack + object on heap
 - The new keyword always allocates memory on the heap
 - Parentheses () after the class name are mandatory in Java (invokes the default constructor)
- Printing an object in Java:
 - System.out.println(obj) prints something like: Calculator@3a5b6c (ClassName@hashcode)
 - The hashcode is an internal identifier used by Java for searching/sorting objects
 - This is not the raw memory address; it is derived via a hashing mechanism
 - Full understanding comes with inheritance and the toString() method (covered later)

Object Allocation, Null References, and NullPointerException

- Declaring a reference without allocating an object:
 - Data obj; → compile error: local variable may not have been initialized
 - Data obj = null; → valid syntax; obj holds memory address 0 (null)
 - Compiler issues a warning: null pointer access
 - Attempting obj.num1 = 1 throws: NullPointerException: cannot assign field num1 because obj is null
 - This confirms: a reference must always point to a valid object before use
- Allocating an object:
 - Data obj = new Data();
 - new Data() allocates an object on the heap at, say, address 800
 - obj (on the stack) now holds 800
 - obj.num1 = 11 is now valid; navigates to address 800 and sets num1

- Default values when an object is allocated on the heap:
 - int, short, long, byte: 0
 - float, double: 0.0
 - boolean: false
 - char: empty (blank character)
 - Any reference type: null
 - These defaults are automatic; no explicit initialization required for data members
- Accessing a method on a null reference:
 - If str is a String reference inside Data and has not been assigned an object, str is null by default
 - Calling obj.str.length() throws NullPointerException: cannot invoke length() because str is null
 - Fix: obj.str = new String("hello"); allocates a String object on the heap; str now points to it
 - Note: strings can also be initialized without new (covered later)
- Arrays as objects:
 - Arrays in Java are treated as objects
 - byte[] arrInput = new byte[100]; allocates an array object on the heap
 - Printing arrInput directly gives its hashCode, not its contents
 - This is why the console input program converts the byte array to a String before printing

Instance Variables, Static Variables, and Initialization Mechanisms

- Instance variables:
 - Belong to an object; require an object to be created before they can be accessed
 - Each object gets its own independent copy
 - Example: obj1.num1 = 1 and obj2.num1 = 2 are completely separate
- Static (class) variables:
 - Declared with the static keyword; allocated at the class level, not per object
 - Only one copy exists regardless of how many objects are created
 - Accessible directly on the class name: Data.iStatic (preferred)
 - Accessing via an object (obj1.iStatic) works but generates a compiler warning
 - Demonstrated via debugging:
 - Before any object is created, Data.iStatic already holds 0 (allocated at class load)
 - Setting obj1.iStatic = 11 immediately changes Data.iStatic and obj2.iStatic to 11 as well
 - They are all the same variable; the object name is just a path to the same class-level copy
- Three places to initialize a data member:
 - At the point of declaration:
 - Best for fixed literal values that are the same for all objects
 - Example: int num1 = 10; directly in the class body

- Cannot contain logic; only literal values
 - Inside an initialization block:
 - An unnamed block of curly braces {} written directly inside the class body
 - Can contain any executable code: if conditions, loops, function calls
 - Use when the initial value depends on some logic
 - Example: `if (today == leapYear) { num1 = 22; } else { num1 = 23; }`
 - A class can have multiple initialization blocks; the compiler merges them in sequence
 - Variables declared inside an init block are local to that block
 - Inside a constructor:
 - Best when each object needs a different initial value supplied by the caller
 - Constructors can take arguments and can be overloaded
 - Overloading rules: different number, type, or sequence of parameters
- Initialization vs. assignment (critical distinction):
 - Initialization: a value is set at the exact moment memory is allocated for the variable
 - Assignment: a value is changed after memory has already been allocated and possibly held another value
 - Example: `int num1 = 1;` inside a constructor is technically assignment (memory already existed with default 0)
 - True initialization in C++ requires the member initializer list: `Data() : num1(80) {}`
 - Java's equivalent is the initialization block or point-of-declaration initialization
 - C++ member initializer list is required for constants; Java's init block serves the same purpose
- Execution sequence per object creation:
 - Data member declarations (and point-of-declaration initializations) are processed
 - Initialization block(s) execute in order of appearance
 - Constructor executes
 - This sequence repeats for every new object created
- Static initialization block:
 - Preceded by the static keyword: `static { ... }`
 - Executes exactly once per class load, not per object
 - Can only access static variables and static methods of the class
 - Attempting to access a non-static member (e.g., `num1 = 1`) inside a static block causes a compile error
 - Demonstrated: even with no objects created, the static block fires when the class is loaded
- Java version note on static blocks:
 - Before Java 7: a static block could run without a `main()` method being present
 - Java 7 and later: JVM requires `public static void main(String[] args)` to be present; without it, throws "main method not found" error even if a static block exists

Constructors: Definition, Rules, and Usage

- Definition: a constructor is a special member function of a class that:
 - Has the same name as the class
 - Has no return type (not even void)
 - Is automatically invoked whenever an object of that class is created using new
- Syntax rules:
 - `public void Data() { }` → this is a regular method named Data, NOT a constructor (has void)
 - `public Data() { }` → this IS a constructor (no return type)
- Auto-invocation:
 - Every time `new Data()` is called, the constructor fires automatically
 - Creating two objects (`new Data(); new Data();`) fires the constructor twice
 - Creating a reference (`Data obj;`) does NOT fire the constructor; no object is created
- Default constructor:
 - If no constructor is defined, Java provides a default no-argument constructor implicitly
 - The `()` in `new Data()` is the invocation of this default constructor; it is mandatory in Java
- Constructor overloading:
 - Constructors can be overloaded just like methods
 - Differentiated by: number of parameters, type of parameters, or sequence of parameter types
 - Allows passing different values for each object at creation time
- When to use a constructor vs. other initialization:
 - Point of declaration: fixed literal, same for all objects
 - Initialization block: logic-based, same for all objects
 - Constructor: dynamic, different per object based on caller-supplied arguments

Language Fundamentals: Conditionals, Loops, and Arrays

- Conditional statements:
 - `if / else if / else`: identical syntax to C++
 - Key Java difference: condition must evaluate to a boolean (true or false)
 - Java does not treat non-zero as true or zero as false (unlike C++)
 - `if (num1 = 10)` is a compile error in Java (assignment inside if is rejected)
 - In C++, this would assign 10 to num1, evaluate as true (positive), and always execute the if block
 - `if (true)` always executes; `if (false)` never executes
 - `boolean flag = true; if (flag) { ... }` is valid and idiomatic Java
 - `else if` and `else` are optional; any combination of `if / else if / else` is valid
 - Equality check: use `==` (double equals); single `=` is assignment
- Switch/case:
 - Identical to C++ in basic syntax: `switch(num1) { case 1: ... break; case 2: ... break; default: ... }`

- break is mandatory to stop fall-through execution; without it, execution continues into the next case
- Curly braces inside a case are optional but required if you want to declare local variables scoped to that case
 - Without braces: local variables declared in one case are in the scope of the entire switch block, causing duplicate variable errors
- default case handles any value not matched by explicit cases
- Java-specific additions (covered later):
 - Java 7+: String values supported in switch
 - Newer Java: switch expressions (a different syntax form)
- Loops:
 - for loop: three parts: initialization; condition; increment
 - None of the three parts are mandatory: for(;;) is an infinite loop
 - The loop variable's scope is limited to the for block if declared inside the initialization
 - Can use a pre-declared variable instead: int num1 = 0; for(; num1 < 10; num1++)
 - The initialization variable, condition variable, and increment variable can all be different
 - while loop: condition only; no built-in initialization or increment
 - Programmer is responsible for incrementing inside the loop body to avoid infinite loops
 - Use when repetition depends purely on a condition, not a counter
 - do-while loop: executes the body first, then checks the condition
 - Guaranteed to execute at least once
 - Syntax: do { ... } while (condition);
 - break: exits the loop immediately, jumps to the statement after the loop
 - continue: skips remaining statements in the current iteration, jumps back to the loop condition
 - All three loop types and break/continue behave identically to C++
- Arrays in Java:
 - An array is a contiguous collection of elements of the same data type
 - Arrays in Java are treated as objects; always allocated on the heap
 - Three syntax styles for single-dimensional arrays:
 - `int[] arr = new int[10];` → allocates 10 integers, all defaulting to 0
 - `int[] arr = new int[]{1,2,3,4,5};` → allocates and initializes inline
 - `int[] arr = {1,2,3,4,5};` → compiler infers `new int[]`; shorthand form
 - All three produce an object on the heap; only the syntax differs
 - Accessing elements: `arr[0] = 1;` (zero-indexed)
 - Printing the array directly gives the hashCode; must iterate to print values
 - Iterating with standard for loop: `for (int i = 0; i < arr.length; i++) { System.out.println(arr[i]); }`
 - `arr.length` gives the size dynamically; avoids hardcoding
 - Enhanced for-each loop (Java 5+): `for (int data : arr) { System.out.println(data); }`

- Cleaner syntax; iterates every element automatically
- Multi-dimensional arrays (terminology only; not implemented today):
 - Matrix array: every row has the same number of columns (e.g., `int[5][10]`)
 - Jagged array: each row can have a different number of columns

Console Input Design, Modularity, and the Setter/Getter Pattern

- Console input flow (step-by-step, for interview recall):
 - Declare a byte array: `byte[] arrInput = new byte[size];`
 - Read from console: `System.in.read(arrInput);` (returns total bytes read including `\n` and `\r`)
 - Create a trimmed array: `byte[] arrFinal = new byte[length - 2];` (removes `\n` and `\r`)
 - On Linux: only `\n` is appended, so use `length - 1`
 - Copy contents: `System.arraycopy(arrInput, 0, arrFinal, 0, length - 2);`
 - Parameters: source array, source start index, destination array, destination start index, number of bytes to copy
 - Convert to String: `String objString = new String(arrFinal);`
 - All input from the console arrives as a String first; convert to target type afterward
 - Conversion uses wrapper class static methods: `Integer.parseInt(str)`, `Float.parseFloat(str)`, etc.
 - Wrapper classes exist for every primitive type; they wrap the primitive in a class and provide helper methods
- Modularity rule (critical design principle):
 - Console I/O (`System.out.print`, `System.in.read`) must never appear inside business logic classes
 - It belongs only in `main()` or dedicated input utility classes
 - Reason: the logic of a Calculator (add, subtract, multiply) does not change across console, GUI, mobile, or web applications
 - If console I/O is embedded in the Calculator class, it cannot be reused in any other platform
 - Keeping business logic clean makes it a reusable library across all four platforms
 - `main()` will be different for each platform; business logic classes remain unchanged
- Reusing a library across projects in Eclipse:
 - Right-click the project that needs the library > Properties > Java Build Path > Libraries > Add Class Folder
 - Select the bin folder of the source project (contains compiled `.class` files, not `.java` source)
 - The library now appears under Referenced Libraries in the project
 - Import and call its classes directly: `int day = ConsoleInput.getln();`
- Getter and setter (accessor/mutator) pattern:
 - Data members declared private cannot be accessed directly from outside the class

- Setter (mutator): a method that validates and sets the value of a private data member
 - Example: `public void setDay(int dd) { if (dd < 1 || dd > 31) { day = 1; } else { day = dd; } }`
- Getter (accessor): a method that returns the value of a private data member
 - Example: `public int getDay() { return day; }`
- Eclipse shortcut: right-click in editor > Source > Generate Getters and Setters
- Benefit: validation logic is written once inside the class; every caller is automatically validated
- Without this pattern, every caller would need to duplicate the validation code

Assignment: Date Utility Program (Due Tomorrow, 11th September)

- Create a new Eclipse project called utility
- Inside it, create a Date class with three private data members:
 - `private int day;`
 - `private int month;`
 - `private int year;`
- Implement a single combined setter: `public void setDate(int dd, int mm, int yyyy)`
 - Validate month first (month affects day validation):
 - If `mm < 1` or `mm > 12`: `month = 1` (default)
 - Else: `month = mm`
 - Validate day based on month:
 - Months with 31 days (1,3,5,7,8,10,12): day must be 1–31
 - Months with 30 days (4,6,9,11): day must be 1–30
 - February (month 2):
 - Leap year: day must be 1–29
 - Non-leap year: day must be 1–28
 - Invalid day: reset to 1
 - Validate year: apply leap year logic where needed
- Implement getters: `getDay()`, `getMonth()`, `getYear()`
- Create a menu-driven program in `main()` with a loop (do-while recommended):
 - Option 1: Set date (prompt user for day, month, year; call `setDate`)
 - Option 2: Add days
 - Prompt for number of days to add
 - Handle month rollovers: e.g., adding 15 days to 10 Sep → 25 Sep; adding 150 days requires month and year changes
 - Cannot produce invalid results like “160 September”
 - Option 3: Add months
 - Convert months to actual days based on the number of days in each month traversed
 - Example: adding 4 months to 10 Sep → count days in Sep (30), Oct (31), Nov (30), Dec (31) = 122 days total
 - Option 4: Add years

- Handle Feb 29 edge case: if current date is 29 Feb 2024 and 3 years are added → 2027 is not a leap year, so date resets to 1 Mar 2027 (or 1 Feb 2027 per chosen logic)
 - Option 5: Display date (print current day/month/year)
 - Option 6: Compare dates (optional; provide the menu option but implementation not required)
 - Option 7: Exit (break out of the loop)
- Link the ConsoleInput library from the previous project via Java Build Path > Add Class Folder (bin of ConsoleInput project)
- Use ConsoleInput.getln() in main() for all user input; no console I/O inside the Date class
- Strict rule: no ChatGPT, Google Gemini, or any AI/search tool; all logic must be original
- Instructor will take random remote screen control to review code; completion is not required but progress is mandatory
- Code shared on chat: Day 4 notes PDF and today's code have been sent to the group

Next Session Preview (11th September, 9:00 AM)

- Review and improve the Date assignment; instructor will check code remotely
- Remaining Java language fundamentals to be covered:
 - Inheritance: concept and implementation in Java
 - Abstract classes
 - Function overriding
 - Defining constants in Java
 - Other core language syntax topics
- Class loaders:
 - How Java loads classes
 - Why static blocks fire at class load time
 - How variable initialization is sequenced during class loading
- Introduction to the Console class (introduced in Java 6)
- Scanner class: deferred until the Java IO package session
- Possible next assignment if all topics are completed on time

Chat with meeting transcript:

<https://notes.granola.ai/t/b87e8eba-7de8-48ce-a96a-9dfbbd8c5a4a-00demib2>